

Pomme de terre

- **Limite de temps** : 10 minutes
- **Environnement** : un GPU (≈16 GB VRAM), sans internet
- **Taille de la solution** : `solution.ipynb` ≤ 1 MB
- **Stockage** : 5 GB

Tâche

Votre ami vous propose de jouer à un jeu de devinettes. En tant que juge, il choisit un mot caché dans un vocabulaire fixe, et vous devez le trouver en au plus 30 tours. À chaque tour, le juge compare deux mots et indique lequel est sémantiquement le plus proche du mot caché. Chaque partie commence par la paire fixe `lamp` vs `potato`, car ce sont deux des choses préférées de votre ami. Votre programme propose ensuite un nouveau mot. Le mot gagnant de la comparaison est conservé et comparé à votre proposition suivante. Vous gagnez une partie dès que vous proposez exactement le mot caché. La comparaison ne tient pas compte de la casse (minuscules/majuscules). Chaque mot que vous proposez doit appartenir à `dataset/vocabulary.json`.

Un exemple complet avec le protocole et le chargement des données se trouve dans `solution.ipynb`. Vous pouvez modifier la classe `PublicEmbeddingPlayer`. Votre programme est initialisé une fois et joue toutes les parties en une seule exécution ; le protocole crée un nouveau `PublicEmbeddingPlayer` au début de chaque partie.

Le juge

Votre programme envoie un objet JSON au juge, et le juge répond avec un objet JSON.

Voici un exemple détaillé, dans lequel le mot caché est indiqué uniquement pour expliquer le protocole :

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}          <- matches: game won
```

Les tours sont numérotés de 1 à 30.

Les valeurs possibles de `verdict` sont `first`, signifiant que `word1` est plus proche, `second`, signifiant que `word2` est plus proche, ou `same`, signifiant que les deux mots sont à égale distance du mot caché.

`winner_word` est le mot conservé pour la comparaison suivante. En cas de verdict `same`, le premier mot reste.

Jeu de données

Commun à chaque partition (split) :

- `dataset/vocabulary.json` — 1602 mots uniques en minuscules. Le mot caché est toujours l'un d'entre eux.
- `dataset/public_embeddings.npy` — `float32`, de forme `(1602, 2560)`. La ligne `i` correspond au mot `i` dans le vocabulaire. Il s'agit d'embeddings *publics* ; le juge utilise une représentation différente, privée.

Les splits sont des ensembles de mots cachés :

Partition	Mots	Réponses	Utilisation
<code>test_public</code>	120	<code>dataset/test_public.json</code>	exécuter votre solution et calculer vous-même son score
<code>test_leaderboard_a</code>	120	cachées	classement en direct
<code>test_leaderboard_b</code>	120	cachées	classement final

Il n'y a pas de `train split` — rien n'est fitted à partir de lignes étiquetées.

Modèles fournis

Deux modèles d'embeddings préentraînés sont fournis dans cette tâche et peuvent être utilisés :

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Tous deux doivent être chargés depuis leur chemin local ; un identifiant de hub Hugging Face tel que "BAAI/bge-m3" déclenche un téléchargement et échoue, car l'évaluation est effectuée hors ligne. Chaque répertoire contient un `example.py` exécutable montrant l'appel hors ligne.

Bibliothèques disponibles : `numpy`, `torch`, `sentence-transformers`. Aucun accès à internet, aucun téléchargement, aucun autre package.

Sortie

Aucune. Il s'agit d'une tâche interactive : votre solution n'écrit aucun fichier de réponse ; elle communique avec le juge via `stdin/stdout` comme décrit ci-dessus.

Métrique

Une partie résolue au tour t obtient un score de $1.0 - 0.02 \times \max(0, t - 10)$; une partie non résolue en 30 tours obtient un score de 0. Ainsi, les tours 1-10 rapportent 1.00, le tour 20 rapporte 0.80, et le tour 30 rapporte 0.60.

Le score de votre tâche est le score moyen des parties $\times 100$, compris entre 0.00 et 100.00.

La limite de 10 minutes constitue un budget unique couvrant le démarrage, la préparation et les 120 parties de l'ensemble de test.

Comment soumettre

1. Ouvrez `solution.ipynb`, modifiez `PublicEmbeddingPlayer` et exécutez toutes les cellules pour vérifier que tout fonctionne.
2. Facultativement, vérifiez-le localement : `python local_test.py solution.ipynb --limit 5`. Le juge local utilise les embeddings *publics*, son score n'est donc qu'indicatif.
3. Enregistrez `solution.ipynb`.
4. Ouvrez l'onglet Git dans la barre latérale gauche de JupyterLab.
5. Ajoutez `solution.ipynb` à la zone staging (l'icône + à côté).
6. Saisissez un message de commit et cliquez sur Commit.
7. Cliquez sur l'icône de nuage avec une flèche vers le haut pour effectuer le push.
8. Revenez sur cette page du concours et cliquez sur Submit, en utilisant le même message de commit que celui que vous avez fourni.

Soumettez exactement un fichier, nommé `solution.ipynb`, couvrant toute préparation et toute inférence nécessaires.